

1.1 — Agentic Loops for Autonomous Task Execution

Concept

When Claude uses tools, it follows a cycle: send a message → Claude decides to call a tool → execute the tool → return the result → Claude decides what to do next. This is the **agentic loop**.

The key mechanism is `stop_reason` :

- `"tool_use"` → Claude wants to use another tool. Keep the loop going.
- `"end_turn"` → Claude is done. Present the response.

Code Example

```
while (true) {
  const response = await client.messages.create({ /* ... */ });

  if (response.stop_reason === 'end_turn') {
    return response; // Done – present to user
  }

  // Execute requested tools
  const toolResults = await executeTools(response.content);
  messages.push({ role: 'assistant', content: response.content });
  messages.push({ role: 'user', content: toolResults });
}
```

Common Mistakes

1. **Parsing text to detect completion** — Don't check if the assistant said "I'm done." Use `stop_reason` .
2. **Arbitrary iteration caps** — Don't set `maxIterations = 5` as the primary stopping mechanism.
3. **Checking for text content** — Don't assume the loop is done because the response contains text.

References

- [Anthropic Agent SDK Documentation](#)

